

CSCE 620 COMPUTATIONAL GEOMETRY · MODULE 12 · LECTURE NOTES

Geometry in Three Dimensions

Collision queries, decomposition, and broken meshes.

MODULE OUTCOMES

- Implement GJK for convex distance and intersection.
- Explain EPA and why penetration depth needs it.
- Apply convex decomposition and say why it is needed.
- Diagnose and repair the standard mesh defects.
- Use the separating axis theorem where it fits.

1 GJK and EPA

GJK REDUCES INTERSECTION TO A QUESTION ABOUT THE ORIGIN

Two convex shapes A and B intersect if and only if their Minkowski difference $A \oplus (-B)$ contains the origin. If some point is in both shapes, its difference with itself is zero; conversely, if the origin is in the difference, some pair of points coincides.

So the problem becomes: is the origin inside this one convex set? Two shapes have become one, and a pairwise question has become a containment question — which is a considerably simpler thing to search for.

And the difference is never actually constructed. GJK requires only a *support function*: given a direction, return the farthest point of the shape in that direction. The support function of the difference is the difference of the support functions, evaluated in opposite directions.

That abstraction is why GJK works on any convex shape — polyhedra, spheres, capsules, cylinders, cones, ellipsoids, and the convex hull of any point set, all behind one interface, with no face lists and no vertex counts. **It is the most reusable idea in this module**, and it is why a single code path handles every collision primitive an engine ships.

```

support(A, B, d) = farthest(A, d) - farthest(B, -d)

simplex = [ support(A, B, arbitrary_direction) ]
d = -simplex[0]

loop:
    p = support(A, B, d)
    if dot(p, d) < 0: return SEPARATED      # EARLY EXIT: a separating
                                           # plane must exist

    simplex.add(p)
    if simplex contains the origin: return INTERSECTING
    d = direction from the simplex toward the origin
    drop simplex vertices that cannot contribute

Converges in a few iterations. Gives DISTANCE free when separated.

```

The early exit is the valuable part in practice. If the farthest point of the Minkowski difference in direction d does not reach past the origin, then no point of the difference does, so a separating plane exists and the shapes cannot intersect. Separated pairs are rejected in one or two iterations, and in a broad-phase-filtered narrow phase most pairs are separated — so the common case is the cheap one.

EPA: GJK TELLS YOU THAT, NOT HOW MUCH

GJK returns a boolean when the shapes overlap, and that is not enough for a physics solver, which needs the penetration depth and the contact normal in order to compute an impulse or a positional correction (CSCE 649 Module 06).

EPA — the Expanding Polytope Algorithm — starts from the simplex GJK terminated with (which encloses the origin) and repeatedly expands it toward the boundary of the Minkowski difference, by finding the face nearest the origin and pushing it outward with a new support point.

When no further expansion is possible, the nearest face gives the penetration depth and the contact normal — the minimum translation that separates the shapes.

EPA is slower and markedly less numerically stable than GJK, because it maintains and repeatedly modifies a polytope near a degenerate configuration. **This is why engines work hard to avoid deep penetration in the first place:** continuous collision detection, speculative contacts, and conservative advancement all exist in large part to keep EPA off the critical path, and to keep penetrations shallow enough that it converges quickly when it does run.

2 The separating axis theorem

	GJK + EPA	SAT
Applicable shapes	Any convex shape, through its support function — including curved ones.	Polyhedra with explicitly known faces and edges.

	GJK + EPA	SAT
How axes are chosen	Iteratively, by search.	Enumerated: both shapes' face normals, plus the cross products of all edge-direction pairs.
Cost for two boxes	Iterative; a few support evaluations.	Exactly 15 axes (3 + 3 face normals, 9 edge cross products), fixed and branch-free — which vectorises and pipelines beautifully.
Distance when separated	Free, if run to convergence.	Not provided.
Penetration depth	Requires EPA.	Falls out directly — the axis of minimum overlap is the answer.
Curved shapes	Works unchanged.	Does not apply — there is no finite axis set.

Use SAT for boxes and simple primitives, and GJK for general convex shapes. The fixed 15-axis test for two oriented bounding boxes is the reason SAT survives despite GJK's greater generality: it is predictable, branch-free, and fast, and box-versus-box is an extremely common query. **Most engines ship both**, with a dispatch table selecting the specialised routine when both shapes are primitives and falling back to GJK otherwise.

3 Convex decomposition

EVERYTHING FAST REQUIRES CONVEXITY; NOTHING AUTHORED IS CONVEX

GJK, SAT, and the Minkowski sum of Module 11 all require convex inputs. So does the guarantee that straight-line motion within a navmesh cell is collision-free. Convexity is the precondition for essentially every fast geometric query in an engine.

And no interesting authored asset is convex — a chair, a weapon, a character, a staircase, a piece of architecture. Concavity is what makes a shape recognisable.

Exact convex decomposition is expensive and produces far too many pieces. A modestly concave mesh can decompose into hundreds of exactly convex parts, and the collision cost scales with the piece count — so an exact decomposition is frequently slower than the problem it solves.

So approximate convex decomposition is used instead: partition the mesh into parts that are *nearly* convex, take the convex hull of each, and accept the small volume error where the hull bulges past the original surface. **V-HACD is the standard implementation, and 20 to 40 pieces is a typical budget** for a character or a prop — chosen by measuring, not by principle.

- **Primitive fitting** — boxes, spheres, and capsules placed by hand or fitted automatically. **By far the cheapest, and still extremely common**, especially for characters, where a capsule is both fast and behaviourally better than an accurate shape.
- **Convex hull of the whole mesh** — one shape, crude, and entirely adequate for debris, pickups, and anything the player will not inspect closely.
- **Triangle soup with a BVH** — exact against the actual surface, and used for **static level geometry only**, because **a triangle soup has no inside**: it can answer 'does this ray hit the surface' but not 'is this point within the object', so a dynamic body can tunnel through or come to rest inside it.
- **Signed distance field** — sampled on a grid, giving cheap inside/outside and gradient queries. **Excellent for deformation, soft bodies, and destruction**, at the cost of memory and of resolution-limited sharp features.
- **Approximate convex decomposition** — the general answer for dynamic concave objects, and the only one that supports accurate contact against arbitrary authored shapes.

4 Broken meshes

Defect	What it breaks	Repair
Duplicate or near-duplicate vertices	Adjacency is destroyed — faces that should share an edge do not, so the mesh behaves as though torn even though it renders correctly.	Weld within a tolerance. Note that this is the non-transitive-equality problem of Module 01 §3, and the tolerance must be chosen with that in mind.

Defect	What it breaks	Repair
Holes	Inside and outside become undefined , so signed distance fields, booleans, volume, and mass properties all fail — silently.	Fill by identifying boundary loops and triangulating them (Module 05).
Non-manifold edges	More than two faces share an edge , so a half-edge representation cannot be built and 'the surface' is not a surface.	Split the edge, or cut the mesh along it into manifold components.
Inconsistent face winding	Normals point inward on some faces, so backface culling, lighting, and orientation tests are wrong.	Flood-fill orientation from a seed face across the adjacency graph.
Self-intersection	Volume is undefined and booleans fail (Module 06).	Genuinely hard, with no cheap fix. Frequently the practical answer is to remesh — voxelise and re-extract — rather than repair. Extremely common in sculpted and scanned assets.
Degenerate faces	Zero area, so the normal is undefined and every downstream predicate is unreliable.	Collapse the degenerate edge and remove the face.

WATERTIGHT IS THE PROPERTY THAT MATTERS

A mesh is watertight (closed and manifold) when every edge is shared by exactly two faces and the face orientations are consistent.

Only then is 'inside' defined — and inside is exactly what signed distance fields, boolean operations, volume, centre of mass, and the inertia tensor (CSCE 649 Module 07) all require.

A mesh with a single small hole has no inside at all, in the strict sense. Every one of those computations will still run and will produce a number; the number will be meaningless. **Silent wrong answers, which Module 01 §3 identified as the worst outcome.**

So validate before trusting. Check the Euler characteristic and the per-edge face counts — a few lines of code that catch most of the table above, and which belong in the asset import path rather than in a debugging session six months later.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Ericson — Real-Time Collision Detection	The reference for this whole module. GJK, SAT, bounding volumes, and the practical choices of §3. Library copy; the author's notes and errata are free.

Resource	What it covers for this module
Casey Muratori — Implementing GJK (free video)	The clearest explanation of §1 available , built up from the support-function idea rather than from the published pseudocode.
Mamou — V-HACD (free)	The approximate convex decomposition of §3, implemented and widely used.
Attene, Campen & Kobbelt — Polygon Mesh Repair: An Introduction (free)	The §4 defect taxonomy , done systematically, with what can and cannot be repaired.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement support functions for a point set, a sphere, and a capsule behind one interface.
- 2 **Implement GJK** for intersection and verify against a brute-force separating-axis search.
- 3 **Count GJK iterations** for separated, touching, and deeply overlapping pairs. Report the three distributions.
- 4 Extend GJK to return the distance between separated shapes.
- 5 Implement EPA and compare its stability at shallow and deep penetration.
- 6 Implement SAT for two oriented boxes with all 15 axes. Benchmark against GJK on the same pairs.
- 7 **Run V-HACD on a concave asset** at several piece budgets. Measure collision cost and volume error against piece count, and plot both.
- 8 **Write a mesh validator** checking the Euler characteristic, edge-to-face counts, winding consistency, and degenerate faces.
- 9 Run it on assets you have and report what fraction are watertight.
- 10 Fill the holes in one and verify that its computed volume becomes meaningful.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why does the Minkowski difference containing the origin mean the shapes intersect?
- 2 What is a support function, and why does it make GJK general?
- 3 Explain GJK's early exit.
- 4 What does EPA provide, and why do engines avoid needing it?
- 5 Compare GJK and SAT on six axes.
- 6 Why is exact convex decomposition impractical?

- 7 Name five collision representations and when each is used.
- 8 Why can a triangle soup not be used for dynamic objects?
- 9 Name six mesh defects and what each breaks.
- 10 Define watertight and say why it matters.